

The Quite OK Video format

a lossless and lossy video compression format

Rene Brokholm · based on QOI (Quite OK Image) and QOA (Quite OK Audio)

Version 3.10 · September 2026 · public domain

1 Overview

QOV is a simple, fast video format designed for real-time performance and ease of implementation. It supports a spectrum of use cases from strictly lossless, fast-decode scenarios to highly compressed lossy video, and streams of unknown length.

- **Fast decoding** — designed for CPU-based decoding without specialized hardware.
- **Simplicity** — a reference decoder fits in roughly 500–1000 lines of code.
- **Flexibility** — lossless RGB/RGBA or YUV 4:4:4 / 4:2:2 / 4:2:0; lossy mode with configurable quality (quantization, 8×8 DCT) at 10–50× compression.
- **Streaming** — unknown-length streams with periodic sync markers.

A QOV file is a sequence of **chunks**: keyframes, P-frames, audio, sync markers, an optional seek index, and an end marker. Pixels are either coded with QOI-style run/length opcodes (RGB and YUV variants) or, in lossy mode, as quantized 8×8 DCT residuals against a predicted reference. The companion format **QOV-S** defines datagram-based live streaming of the same chunks.

2 File Header

The file starts with a 24-byte header (versions 0x01/0x02) or a 32-byte header (version 0x03, lossy). All integers are big-endian.

Offset	Size	Name	Description
0	4	magic	Magic bytes "qovf" (0x716f7666)
4	1	version	Format version (0x01, 0x02 or 0x03)
5	1	flags	Feature flags bitfield (§2.3)
6	2	width	Video width (1–65535)
8	2	height	Video height (1–65535)
10	2	frame_rate_num	Frame rate numerator
12	2	frame_rate_den	Frame rate denominator
14	4	total_frames	Total frame count (0 = unknown / streaming)
18	1	audio_channels	Audio channels (0 = no audio, 1–8)
19	3	audio_rate	Audio sample rate (0–16777215 Hz)
22	1	colorspace	Color space identifier (§2.5)

Offset	Size	Name	Description
23	1	quality	Quality level 0–100. 0 = lossless, 100 = near-lossless

2.1 Extended Header (Version 0x03)

If **version** is **0x03**, the header is 32 bytes long:

Offset	Size	Name	Description
24	1	y_quant_base	Y plane base quantization (1–64, or 0 to derive)
25	1	uv_quant_base	UV plane base quantization (1–64, or 0 to derive)
26	1	temporal_thresh	Temporal similarity threshold (0–32, or 0 to derive)
27	1	dct_qp_base	Base QP for DCT blocks (0–51, or 0 to derive)
28	4	reserved	Must be 0x00000000

2.2 Versions

Value	Name	Description
0x01	Q0V_V1	DEPRECATED (v3.10). Encoders MUST NOT emit; decoders MUST reject
0x02	Q0V_V2	DEPRECATED (v3.10). Encoders MUST NOT emit; decoders SHOULD still accept
0x03	Q0V_V3	Current. The only version encoders produce

Single-version profile (normative, v3.10): encoders emit version 0x03 only — including for strictly lossless content, which is a v3 file with LOSSY_MODE/DCT_ENABLED unset and quality 0. v3 fully subsumes v2: the chunk streams are identical, only the 8-byte lossy header extension differs. Decoders MUST reject version 0x01 and MUST accept 0x03; the v2 read path is retained for existing files.

2.3 Flags Byte

Bit	Name	Description
0	HAS_ALPHA	Frames include an alpha channel
1	HAS_MOTION	Motion vectors enabled for P-frames
2	HAS_INDEX	Index table present at the end of the file
3	EXP_GOLOB	DCT blocks use Exp-Golomb coefficient coding (v3.5+)
4	INTRA_REFRESH	P-frames carry rolling intra refresh bands (v3.4+)
5	LOSSY_MODE	Lossy encoding enabled (version 0x03+)
6	DCT_ENABLED	DCT block encoding available (version 0x03+)
7	INTRA_DCT_KF	Lossy keyframes use intra DCT blocks (v3.3+)

Bits 3 and 4 were previously named HAS_BFRAMES and ENHANCED_COMP; neither feature ever shipped and the bits are reclaimed by EXP_GOLOB (v3.5) and INTRA_REFRESH (v3.4). Files setting them with the old meanings do not exist.

2.4 Chunk structure

All chunks share a common header. In version 0x01 the **chunk_size** field is 16-bit (8-byte header, 65 KB limit); in versions 0x02/0x03 it is 32-bit (10-byte header, large frames).

Offset	Size	Name	Description
0	1	chunk_type	Chunk type identifier (§2.4.1)
1	1	chunk_flags	Chunk-specific flags (§2.4.2)
2	2/4	chunk_size	Size of the data after the header, big-endian
—	4	timestamp	Timestamp in microseconds, big-endian

2.4.1 Chunk Type IDs

Value	Name	Description
0x00	SYNC	Sync marker (streaming recovery point)
0x01	KEYFRAME	I-frame (complete image)
0x02	PFRAME	P-frame (references the previous frame)
0x03	BFRAME	B-frame (bidirectional reference)
0x10	AUDIO	Audio data (QOA, or Opus via the flags byte)
0xF0	INDEX	Seek index table
0xFF	END	End of stream marker

2.4.2 Chunk Flags and Compression

Bit	Name	Description
0	YUV_MODE	Frame uses YUV plane-based encoding
1	HAS_MOTION	Frame includes a motion vector block (§7.2)
2	STRUCTURED	DCT P-frame payload uses the structured grammar (v3.9, §5.6)
3	RANGE	Chunk data is adaptive range-coder compressed (v3.8, §2.4.3)
4	COMPRESSED	Chunk data is LZ4 block compressed
5	DCT_BLOCKS	Frame uses DCT block encoding; also marks intra DCT keyframes
6	EXP_GOLOB	DCT coefficient sections are Exp-Golomb coded (v3.5, §5.5)
7	REFRESH_BAND	P-frame payload starts with a refresh band byte (v3.4, §5.4)

If **COMPRESSED** is set, the payload is strictly [uncompressed_size (4 bytes)] [LZ4 block]. If **RANGE** is set, the payload is strictly [uncompressed_size (4 bytes)] [range-coded stream]; the uncompressed_size field is included in chunk_size. RANGE and COMPRESSED are mutually exclusive. Encoders set RANGE instead of COMPRESSED on DCT chunks when range coding is enabled.

2.4.3 Order-0 Adaptive Range Coder (v3.8)

A byte-oriented adaptive range coder (LZMA family: 32-bit range, 64-bit low with a carry cache). It replaces LZ4 on DCT chunks because DCT coefficient bytes are strongly skewed — exactly what an adaptive model exploits and LZ4 cannot.

- **Model.** 256 symbols, each with frequency 1 (total 256). After coding a byte, `freq[s] += 32; total += 32`. When `total > 65536`, all frequencies are halved (`(freq[i] + 1) >> 1`, floored at 1). Encoder and decoder run identical model updates.
- **Coding a byte.** Standard range subtraction: `range /= total`, the symbol located by cumulative-frequency scan; renormalize while `range < TOP` (`TOP = 1<<24`) by shifting a byte of low out through the carry cache.
- **Carry handling.** A `0xFF` run followed by a carry becomes `0x00` bytes and increments the cached byte (cacheSize mechanism).
- **Flush.** Five bytes of low are emitted so the decoder's code register is fully populated for every coded symbol. Decoders MUST tolerate a stream that ends once `uncompressed_size` bytes are decoded; trailing flush bytes are not consumed.

2.5 Colorspace Byte

Value	Name	Description
0x00	SRGB	sRGB, 8-bit RGB (QOI compatible)
0x01	SRGBA	sRGB + alpha, 8-bit RGBA
0x02	LINEAR	Linear RGB, 8-bit
0x03	LINEAR_A	Linear RGBA, 8-bit
0x10	YUV420	YCbCr 4:2:0, 8-bit (best compression)
0x11	YUV422	YCbCr 4:2:2, 8-bit
0x12	YUV444	YCbCr 4:4:4, 8-bit
0x13	YUVA420	YCbCr 4:2:0 + alpha, 8-bit

3 Video Opcodes: RGB Mode (QOI-Compatible)

Used when the colorspace is RGB/RGBA and the **YUV_MODE** chunk flag is 0. Pixels are coded one at a time against a running previous pixel and a 64-entry color index cache, exactly like QOI.

QOV_OP_INDEX — 2-bit tag 00, 6-bit index

```
+-----+-----+
| 2 bits | 6 bits |
|  00   | index |
+-----+-----+
index = (r * 3 + g * 5 + b * 7 + a * 11) % 64
```

QOV_OP_DIFF — 2-bit tag 01, three 2-bit deltas

```
+-----+-----+-----+-----+
| 2 bits | 2 b | 2 b | 2 b |
| 01    | dr | dg | db |
+-----+-----+-----+-----+
bias +2 on each channel; dr, dg, db in [-2..1]
```

QOV_OP_LUMA — green delta plus packed red/blue deltas

```
+-----+-----+-----+
| 2 bits | 6 bits | 8 bits |
| 10    | dg  | dr | db |
+-----+-----+-----+
byte 2 packs dr_dg (4 bits) | db_dg (4 bits)
dg in [-32..31]; dr_dg, db_dg in [-8..7]
```

QOV_OP_RUN — 2-bit tag 11, 6-bit run length

```
+-----+-----+
| 2 bits | 6 bits |
| 11    | run  |
+-----+-----+
run length 1..62
```

QOV_OP_RGB / QOV_OP_RGBA — full literals

```
+-----+-----+-----+-----+
| 11111110 | r | g | b | QOV_OP_RGB (0xFE)
+-----+-----+-----+-----+
+-----+-----+-----+-----+
| 11111111 | r | g | b | a | QOV_OP_RGBA (0xFF)
+-----+-----+-----+-----+
```

4 Video Opcodes: YUV Mode (Plane-Based)

Used when **YUV_MODE** is 1. The planes (Y, U, V, then A when present) are coded sequentially with the same opcode shapes, applied to single bytes instead of pixels.

YUV keyframe opcodes

0x00-0x3F	YUV_INDEX	00 index (6 bits)	hash: (value * 3) % 64
0x40-0x4F	YUV_DIFF	0100 d (4 bits)	bias +8
0x80-0xBF	YUV_LUMA	10 d (6 bits)	bias +32
0xC0-0xFD	YUV_RUN	11 run (6 bits)	
0xFE	YUV_FULL	11111110 value	

Critical: the 64-entry index cache **MUST** be initialized to -1 (not 0) at the start of each keyframe in YUV mode. INDEX is pure retrieval — the cache is updated only by DIFF, LUMA and FULL (each writes $\text{cache}[(\text{value} * 3) \% 64] = \text{value}$). The keyframe coder has no SKIP_LONG opcode, so byte 0x00 = INDEX[0] is valid here and always refers to the value 0 (its hash is slot 0).

5 Temporal Opcodes (P-Frames)

P-frames code each pixel/plane byte against the reference frame (or plane). Unchanged regions are skipped in runs; changed regions use temporal deltas against the reference value.

RGB P-frame opcodes

0x00	SKIP_LONG	00000000 count (16 bits)
0xC0-0xFD	SKIP	11 count (6 bits) count 1..62
0x40-0x7F	TDIFF	01 dr dg db bias +2
0x80-0xBF	TLUMA	10 dg dr-dg db-dg
0xFE	RGB	11111110 r g b
0xFF	RGBA	11111111 r g b a

YUV P-frame opcodes

0x00	YUV_SKIP_LONG	00000000 count (16 bits)
0xC0-0xFD	YUV_SKIP	11 count (6 bits)
0x01-0x3F	YUV_INDEX	00 index (6 bits) index 0 FORBIDDEN
0x40-0x4F	YUV_TDIFF	0100 d (4 bits)
0x80-0xBF	YUV_TLUMA	10 d (6 bits)
0xFE	YUV_FULL	11111110 value

In YUV P-frames byte 0x00 is SKIP_LONG; INDEX[0] therefore cannot be used and value 0 must be coded as FULL.

6 Lossy Extensions (Version 0x03)

If LOSSY_MODE is enabled, additional opcodes are available. The opcodes of this section (0x50–0x53 and 0x58–0x59) take precedence over the base ranges of §5 when the corresponding feature is in use.

6.1 Lossy Skip (Similarity)

SKIP_SIMILAR — skip pixels close enough to the reference

0x58	SKIP_SIMILAR	01011000 count (1b) threshold (1b)
0x59	SKIP_SIMILAR_LONG	01011001 count (16b) threshold (1b)
skip count pixels where the difference is <= threshold		

6.2 DCT Block Encoding

Enabled via the **DCT_BLOCKS** chunk flag. Operates on 8×8 blocks in Y, U, V plane order (then A if present), each plane signalled with its own opcode. Plane geometry follows the header colorspace with odd sizes rounded up (4:2:0 chroma is $\text{ceil}(w/2) \times \text{ceil}(h/2)$; the alpha plane uses luma dimensions and the luma quantizer).

DCT plane opcodes

0x50	DCT_Y	01010000 [DCT data]
0x51	DCT_UV	01010001 [DCT data]
0x52	DCT_SKIP	01010010 count skip count 8x8 blocks (copy reference)
0x53	DCT_ZERO	01010011 count count blocks with zero residual

Each coded block's **DCT data** is:

- **qp_delta** (1 byte): bias-64 delta of the block's absolute QP from the header base QP — | 0 | delta (7 bits) |.
- **DC coefficient** (2 bytes): 16-bit signed big-endian.
- **AC coefficients**, run-level coded in zigzag scan: pairs of | run (4b) | level_size (4b) | followed by the level as a big-endian two's-complement integer of level_size bytes (1–4). The pair 0x00 is EOB (end of block) and is written unconditionally; 0xF0 is a zero run of 16.

Encoder-side normative rules (required for multi-encoder bit-exact parity): a block whose sum of absolute residuals against the reference is $< 32 + 8 \times \text{qp_base}$ is coded as DCT_SKIP; an AC coefficient with $\text{prod} = \text{coeff} \times \text{scale} / \text{quant}[c]$ satisfying $-0.75 < \text{prod} < 0.75$ is coded as level 0 (dead-zone; the DC coefficient is never dead-zoned), otherwise $\text{level} = \text{round}(\text{prod})$. The encoder's local reconstruction **MUST** apply the same rules so its reference matches the decoder's. Zigzag order, quantization tables and the QP-to-scalefactor mapping are implementation-defined: encoder and decoder must agree on them out of band.

6.3 Intra DCT Keyframes

A KEYFRAME chunk with DCT_BLOCKS set carries intra-coded 8×8 blocks instead of the §4 opcode stream — raster order, reconstruction of earlier blocks predicting later ones. **DC prediction** is a fixed rule: the mean of the reconstructed left column and/or top row (clipped to the plane), the rounded average of the two means when both exist, and 128 with no neighbor; all averaging is integer round-half-up. The residual (source – predictor) goes through the §6.2 block format with the same skip rule; reconstruction clamps to [0, 255] and the reconstructed planes become the P-frame reference.

6.4 Intra Refresh Bands (v3.4)

A PFRAME chunk with the REFRESH_BAND flag carries one extra byte at the very start of the payload — before any motion vector data, after decompression:

Refresh band payload layout

```
payload := band_index (1 byte) [ mv_block ] [ plane block streams... ]
band rows = [ rows * band_index / 12 , rows * (band_index + 1) / 12 )
rows = ceil(plane_height / 8) REFRESH_BANDS = 12
```

Blocks whose row lies inside the band follow the §6.3 intra semantics (prediction from already-reconstructed pixels of the current frame), so fresh data never copies a damaged reference; every other block stays inter. A single DCT_SKIP run MAY straddle a band boundary — each block's semantics follow its own row. The whole frame is refreshed at most once per 12 P-frames. Decoders that do not implement this section **MUST** reject PFRAME chunks carrying bit 7.

6.5 Exp-Golomb Coefficient Coding (v3.5)

A DCT chunk MAY set EXP_GOLOB: only the coefficient section of each coded block is replaced by a bit-packed string — everything else (QP delta, opcodes, skip runs, end marker) is unchanged. The section is: $\text{se}(\text{DC})$; zero or more $\text{ue}(\text{zero_run}) / \text{se}(\text{level})$ pairs in zigzag order; a terminating $\text{ue}(64 - k)$ sentinel; zero bits pad to the next byte boundary so the next block's opcode stays byte-aligned. Codes are MSB-first:

Exp-Golomb code definitions

```
ue(v):  write v + 1 in binary, prefixed with bitlength(v + 1) - 1 zero bits
se(v):  v > 0  ->  ue(2v - 1)
        v <= 0 ->  ue(-2v)
```

Quantization, dead-zone, reconstruction and decoded pixels are identical to the byte coding; a conforming encoder MUST produce bitstreams that decode to the same pixels in either coding. Decoders MUST implement both.

6.6 Structured P-frame Grammar (v3.9)

A lossy DCT PFRAME chunk MAY set STRUCTURED to replace the per-block framing of §6.2 with a plane-structured grammar: per-block opcodes, per-block QP deltas and SKIP opcodes are dropped; coefficient sections, quantization, dead-zone and decoded pixels are identical. After the optional band byte and motion vectors, the payload is exactly one section per plane, in Y, U, V (then A) order:

Structured plane section

```
plane_section := qp_delta (1 byte)
                [ run_count (1+ bytes) coded_section ] ...
                run_count (1+ bytes)          ; plane terminator
coded_section := DC (u16 BE) AC pairs ... EOB      ; or the Exp-Golomb variant
```

- **run_count** is the number of blocks skipped before the next coded section; 0 means a coded section follows immediately; 255 means "255 skipped, the chain continues with the next byte". The plane's final chain byte is always < 255 (0 when the plane ends on a coded block) and terminates the plane — no end marker or block count is needed.
- Skip semantics are unchanged: outside the refresh band a skipped block copies the reference; inside the band it fills with the DC prediction. A chain MAY straddle a band boundary.
- Measured on 30 s of real webcam footage: P-frame bytes -43.8% at q60, -47.8% at q30, -34.9% at q85, bit-identical decoded pixels.

v3.9 decoder fix (both grammars): the EOB is written unconditionally, including when the last coded AC lands exactly on zigzag index 63 — a decoder whose AC loop stops at $k = 64$ without consuming the EOB desynchronizes the plane. Skip-chain byte 255 accumulates and continues; decoders MUST accumulate every chain byte.

7 Lossy Quality and Quantization

In lossy mode, pixel data can be quantized before encoding. The parameters below are **derived, not stored**: they are computed from the quality byte with the normative formula (§7.1, integer division, truncation toward zero). If header bytes 24–27 are zero, the decoder derives them from the quality byte.

```
void derive_lossy_params(int quality, lossy_params_t *params) {
    params->y_quant      = CLAMP(1 + (100 - quality) / 8, 1, 64);
    params->uv_quant     = CLAMP(2 + (100 - quality) / 4, 1, 64);
    params->temporal_thresh = CLAMP((100 - quality) / 12, 0, 32);
    params->dct_qp        = CLAMP(51 - (quality * 51 / 100), 0, 51);
}
```

Quality	Y Quant	UV Quant	Temporal Thresh	DCT QP	Typical Ratio
100	1	2	0	0	1.5–3×
85	2	5	1	8	3–6×
50	7	14	4	26	12–20×
30	9	19	5	36	20–35×

Mid-stream quality changes (adaptive streaming, v3.6): encoders MAY change the working quality at any frame boundary. The header always describes the start-time quality; a change becomes decoder-visible only through the bitstream — each coded DCT block’s qp_delta carries the bias-64 delta from the header base QP, and simple-mode quantized values are stored directly. The lossy/lossless mode is fixed by the header and MUST NOT change mid-stream.

8 Frame and Packet Reference

8.1 Keyframe (I-Frame)

Type 0x01. Full pixel data; YUV plane order is Y, then U, then V (then A). The payload ends with the 8-byte end marker 00 00 00 00 00 00 00 01.

8.2 P-Frame and Motion Vectors

Type 0x02. If chunk flag bit 1 (HAS_MOTION) is set, the payload starts with a **motion vector block**, followed by the normal P-frame opcode stream. The MV block appears at most once per chunk, before all pixel and plane streams.

Motion vector block

```
block_size_id (1 byte)  0 = 8x8    1 = 16x16    2 = 32x32    3 = 16x16 half-pel
mv_count (2 bytes, big-endian)
vectors (2 bytes each): mv_x (1 byte, signed)  mv_y (1 byte, signed)
```

- The block grid covers the frame row-major: $\text{grid} = \text{ceil}(\text{dim} / B)$. The first mv_count blocks carry the written vectors; every grid block after mv_count defaults to vector (0, 0).
- Vectors are integer-pel in the range ± 127 pixels. The vectors define the effective reference: each block’s predictor is the previous frame copied from $(x + \text{mv}_x, y + \text{mv}_y)$ with source coordinates **clamped** to the frame borders (never wrapped). All P-frame coding — skips, TDIFF/TLUMA, SKIP_SIMILAR and DCT residuals — applies against this compensated reference.
- block_size_id 3 (v3.6)** selects 16×16 blocks with half-pel-unit vectors: each written byte is the signed displacement $u = 2 \cdot \text{mv}_x + h_x$ (range ± 127 , integer component limited to $-64..+63$). Decoding: $\text{mv}_x = u \gg 1$ (arithmetic), $h_x = u \& 1$. A half-pel offset samples the reference bilinearly, border-clamped, integer-only: horizontal $(a+b+1) \gg 1$, vertical $(a+c+1) \gg 1$, diagonal $(a+b+c+d+2) \gg 2$. Blocks with even u and v compensate exactly as in integer-pel mode. Half-pel mode is a lossy-YUV encoder option; lossless P-frames always compensate with exact copies.
- Chroma** uses the luma vector field at chroma resolution: $\text{mv}_c = \text{mv}_\text{luma} \gg 1$ per subsampled axis (4:2:0 and YUVA420 both axes; 4:2:2 horizontal only; 4:4:4 unchanged). In half-pel mode the chroma shift is $u \gg 2$ per subsampled axis, i.e. chroma stays whole-pel and the fractional luma bit is dropped. The alpha plane uses luma vectors at luma resolution.

- Header flag HAS_MOTION (bit 3) declares that a file uses motion vectors; individual P-frames still choose per chunk via chunk flag bit 1.

8.3 Audio

Type 0x10 with the ordinary chunk header. The payload codec is selected by the AUDIO flags byte (video chunk flag bits do not apply); AUDIO chunks are never LZ4-compressed.

flags	Payload
0x00	QOA (default): one or more QOA frames back to back
0x01	Opus: exactly one Opus packet (Opus carries 48 kHz internally)
other	reserved — decoders MUST skip the chunk gracefully

A QOA payload is a standard QOA frame: 8-byte frame header (channels 1b, sample rate 3b, samples per channel 2b, frame size 2b, big-endian), 16 bytes of LMS state per channel, then 8-byte slices interleaved per channel. For interactive speech, encoders SHOULD capture 16 kHz mono; QOA frames are 256 samples per channel (≈ 16 ms at 16 kHz) independent of the rate. Video and audio timestamps share one monotonic microsecond clock sampled at capture; decoders resample to their playout clock and MUST tolerate a few ms of jitter.

8.4 Sync Marker

Type 0x00, size 8: the bytes "QOVS" followed by a 4-byte frame number.

8.5 Index Table

Type 0xF0, located at the end of the file immediately before the END chunk: entry_count (4 bytes) followed by entry_count entries of [frame_num (4b), offset (8b), timestamp (4b)] — 16 bytes per entry — for all keyframes. Entry offsets point at the SYNC chunk preceding each keyframe.

9 Pseudocode Reference: Decoder Loop

```

while (p < file_size) {
    chunk_type = data[p];
    chunk_flags = data[p + 1];
    chunk_size = read_u32(data + p + 2);    // v2/v3; v1 uses u16
    timestamp  = read_u32(data + p + 6);

    if (chunk_flags & RANGE) {              // v3.8, section 2.4.3
        uncompressed_size = read_u32(data + p + 10);    // included in chunk_size
        payload = range_decode(data + p + 14, uncompressed_size);
    } else if (chunk_flags & COMPRESSED) {
        uncompressed_size = read_u32(data + p + 10);    // included in chunk_size
        payload = lz4_decompress(data + p + 14, uncompressed_size);
    } else {
        payload = data + p + 10;
    }

    switch (chunk_type) {
        case KEYFRAME: decode_keyframe(payload, chunk_flags); break;
        case PFRAME:   decode_pframe(payload, chunk_flags);   break;
        case AUDIO:    decode_audio(payload);                  break;
    }
    p += header_size + chunk_size;
}

```

10 License

This specification is placed in the public domain.

11 Changelog

Version	Summary
3.10	Single-version profile: encoders emit only v3 (lossless = quality 0, LOSSY_MODE off). v1 deprecated — encoders MUST NOT emit, decoders MUST reject; v2 deprecated but readable. 16-bit parse paths removed from the C and TypeScript decoders.
3.9	Structured P-frame grammar (STRUCTURED, bit 2): one qp byte per plane and bare run-chain / coded-section alternation; P-frame bytes −43.8% at q60, bit-identical pixels. Decoder fix: EOB consumed even when the last AC lands on zigzag 63; 255-chains accumulate every byte.
3.8	Order-0 adaptive range coder (RANGE, bit 3) replaces LZ4 on DCT chunks: video bytes −18.6%, wire 647.5 → 541.9 kbps at call settings.
3.7	AUDIO chunk codec flags: 0x00 QOA, 0x01 Opus (one packet), others reserved. 16 kHz mono speech capture recommended; capture-clock semantics defined.
3.6	Mid-stream quality changes (qp_delta rides the bitstream); qov_drop_reference. Half-pel motion refinement: block_size_id 3, u = 2·mv_x + h_x, bilinear border-clamped compensation, chroma whole-pel via u >> 2.
3.5	Optional Exp-Golomb coefficient coding (EXP_GOLOB, bit 6); header bit 3 renamed from the never-implemented HAS_BFRAMES.
3.4	Intra refresh bands (REFRESH_BAND, bit 7) with a leading band byte; header bit 4 renamed from the never-implemented ENHANCED_COMP.
3.3	Intra DCT keyframes: DC-predicted 8×8 blocks through the 3.4.2 block format; header flag INTRA_DCT_KF (bit 7).

Version	Summary
3.2	Motion vectors become normative: MV block placement, row-major grid with implicit (0,0) tail, clamped borders, compensated-reference rule, chroma derivation, ± 127 range.
3.1	Errata: quality table aligned with the normative derivation; decoder pseudocode skips the 4-byte <code>uncompressed_size</code> field; QOA frame layout corrected; DCT plane ordering and geometry specified.